

A10_CVE-2024-42352

Here is a snippet of code written in typescript. I want to check for any potential security vulnerabilities.

```
import { basename } from 'pathe'
import { getIcons } from '@iconify/utils'
import { consola } from 'consola'
import { useAppConfig, defineCachedEventHandler } from 'nitropack/runtime'
import type { NuxtIconRuntimeOptions } from '../schema-types'
import { collections } from '#nuxt-icon-server-bundle'

const warnOnceSet = /* #__PURE__ */ new Set<string>()

const DEFAULT_ENDPOINT = 'https://api.iconify.design'

export default defineCachedEventHandler(async (ctx) => {
  const url = ctx.node.req.url
  if (!url)
    return

  const options = useAppConfig().icon as
  NuxtIconRuntimeOptions
  const collectionName =
  ctx.context.params?.collection?.replace(/\.json$/, '')
  const collection = collectionName
    ? await collections[collectionName]?.()
    : null

  const apiEndPoint = options.iconifyApiEndpoint ||
  DEFAULT_ENDPOINT
  const apiUrl = new URL(basename(url), apiEndPoint)
  const icons = apiUrl.searchParams.get('icons')?.split(',')

  if (collection) {
    if (icons?.length) {
      const data = getIcons(
        collection,
        icons,
```

```

    )
    consola.debug([Icon] serving ${icons ||
[]).map(i => '' + collectionName + ':' + i +
'').join(',')}) from bundled collection)
    return data
  }
}
else if (import.meta.dev) {
  // Warn only once per collection, and only with the
  default endpoint
  if (collectionName && !warnOnceSet.has(collectionName)
&& apiEndPoint === DEFAULT_ENDPOINT) {
    consola.warn([
      [Icon] Collection \${collectionName}\ is not
      found locally,
      We suggest to install it via \npm i -D @iconify-
      json/\${collectionName}\ to provide the best end-user
      experience.,
    ].join('\n'))
    warnOnceSet.add(collectionName)
  }
}

if (options.fallbackToApi) {
  consola.debug([Icon] fetching ${icons ||
[]).map(i => '' + collectionName + ':' + i +
'').join(',')}) from iconify api)
  const data = await $fetch(apiUrl.href)
  return data
}
})

```

Here's a security analysis of the provided TypeScript code. Below are potential vulnerabilities, risks, and best practices to mitigate them:

1. Insufficient Input Validation for URL Parsing

- **Issue:** The code directly uses `ctx.node.req.url` to construct a new `URL` object and fetch resources dynamically.

- **Risk:** If this input is not properly sanitized, it may allow **Server-Side Request Forgery (SSRF)**. An attacker could manipulate URLs to access internal services or sensitive endpoints (e.g., `localhost`, metadata services in cloud environments).
 - **Mitigation:**
 - Validate and restrict the URLs being constructed.
 - Use `allowlist` or `blocklist` patterns to prevent misuse.
-

2. Untrusted Dynamic Imports / Collections Handling

- **Issue:** The code dynamically accesses collections using:

typescript

```
const collection = collectionName ? await collections[collectionName]?.() : null;
```

- **Risk:** This could lead to **Prototype Pollution** or misuse if `collectionName` is not properly controlled. The collection object might be vulnerable to injection attacks.
 - **Mitigation:**
 - Ensure `collectionName` is sanitized and validated before accessing collections.
 - Use a strict schema or allowlist of expected collection names.
-

3. Potential Information Leakage in Development Mode

- **Issue:** The following warning leaks sensitive information in development mode:

typescript

```
console.warn(`[Icon] Collection \`${collectionName}\` is not found locally`);
```

- **Risk:** Exposing internal configuration details or file structures can assist attackers in finding vulnerabilities.

- **Mitigation:**
 - Ensure sensitive logs are not accessible outside of development environments.
 - Use environment variables to toggle logs securely.
-

4. External API Fetch without Proper Error Handling

- **Issue:** The `$fetch` call fetches from a potentially user-controlled endpoint.

```
typescript

const data = await $fetch(apiUrl.href);
```

- **Risk:** If the API endpoint is compromised or the response is malicious, it may introduce **Remote Code Execution (RCE)** or **data integrity issues**.
 - **Mitigation:**
 - Implement timeouts, retries, and error handling for `$fetch`.
 - Consider validating the response schema to ensure it matches expected formats.
 - Add Content Security Policy (CSP) headers to limit script execution.
-

5. Lack of Rate Limiting for API Calls

- **Issue:** The API call can be triggered multiple times without any rate limiting or cache management, especially in this block:

```
typescript

const data = await $fetch(apiUrl.href);
```

- **Risk:** This could lead to **Denial of Service (DoS)** if the endpoint or external API is overwhelmed with requests.
- **Mitigation:**
 - Introduce rate limiting or caching mechanisms to reduce redundant requests.

- Use `defineCachedEventHandler` to cache results effectively.
-

6. Potential Prototype Pollution via Search Params

- **Issue:** The use of `apiUrl.searchParams.get('icons')?.split(',')` could allow unexpected input that might modify critical internal objects if not sanitized.
 - **Risk:** An attacker could attempt to inject malicious payloads into search parameters leading to **Prototype Pollution** or unexpected behavior.
 - **Mitigation:**
 - Sanitize and validate query parameters before processing.
 - Use strict input validation on `icons` values.
-

Summary of Key Recommendations:

- **Input Validation & Sanitization:** Ensure inputs such as `url`, `collectionName`, and query parameters are validated.
- **SSRF Prevention:** Validate and restrict the construction of dynamic URLs.
- **Secure Logging:** Limit logs in non-development environments to avoid information leakage.
- **Error Handling for API Calls:** Implement proper error handling for external fetches.
- **Rate Limiting:** Add rate limiting or caching to prevent excessive API calls.

This analysis outlines several potential risks and security best practices. Make these adjustments to the code to minimize vulnerabilities and ensure safer execution.

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Below is a detailed breakdown with **line numbers** of the vulnerable parts of the code along with explanations:

1. Insufficient Input Validation for URL Parsing (SSRF)

Code:

```
typescript

 9  const url = ctx.node.req.url
10  if (!url)
11    return
...
18  const apiUrl = new URL(basename(url), apiEndPoint)
```

- **Line 9-11, 18:**
 - The `url` is extracted from `ctx.node.req.url` (line 9), which is user-controlled input.
 - The code uses this input directly to construct a `URL` object (line 18). If not sanitized, it could allow **SSRF** (Server-Side Request Forgery) attacks.

Mitigation:

- Validate the `url` input to prevent access to internal or unexpected domains.
-

2. Untrusted Dynamic Access to Collections (Prototype Pollution Risk)

Code:

```
typescript

12  const collectionName = ctx.context.params?.collection?.replace(/\.json$/, '')
13  const collection = collectionName
14    ? await collections[collectionName]?.()
15    : null
```

- **Line 12-15:**

- The collection is accessed dynamically based on `collectionName`, which is derived from user-controlled input.
- If `collectionName` is manipulated, it could lead to **Prototype Pollution** or unexpected behaviors, especially if an attacker attempts to access unintended keys or objects.

Mitigation:

- Use strict validation on `collectionName`. Consider using an **allowlist** of valid collection names.
-

3. Potential Information Leakage in Development Mode

Code:

typescript

```
23 if (import.meta.dev) {
24   if (collectionName && !warnOnceSet.has(collectionName) && apiEndPoint ===
DEFAULT_ENDPOINT) {
25     consola.warn([
26       `[Icon] Collection \`${collectionName}\` is not found locally`,
27       `We suggest to install it via \`npm i -D @iconify-
json/${collectionName}\` to provide the best end-user experience.`,
28     ].join('\n'))
29     warnOnceSet.add(collectionName)
30   }
31 }
```

- **Line 23-31:**
 - This block prints warnings containing sensitive internal information, such as the `collectionName` that was not found locally. This information could be leveraged by attackers in **development mode**.

Mitigation:

- Ensure these logs are disabled in production environments or contain no sensitive details.

4. External API Fetch without Proper Error Handling (Potential RCE Risk)

Code:

typescript

```
32 if (options.fallbackToApi) {
33   consola.debug(`[Icon] fetching ${icons || []}.map(i => '' + collectionName
+ ':' + i + '').join(',') from iconify api`)
34   const data = await $fetch(apiUrl.href)
35   return data
36 }
```

- **Line 32-36:**
 - The `$fetch` method is used to fetch data from an external API. However, there is **no error handling** (like `try-catch`) if the request fails or the response contains unexpected data.
 - An attacker might manipulate the response or endpoint to introduce malicious payloads, leading to **Remote Code Execution (RCE)** or integrity issues.

Mitigation:

- Add error handling (e.g., `try-catch`) around the `$fetch` call.
- Validate the structure of the fetched response.

5. Lack of Rate Limiting for API Calls (DoS Risk)

Code:

typescript

```
34 const data = await $fetch(apiUrl.href)
```

- **Line 34:**

- Since this fetch request can happen dynamically for every request, it could allow **Denial of Service (DoS)** attacks through excessive requests to the external API.

Mitigation:

- Implement **rate limiting** or **caching** to avoid excessive calls to the external API.
-

6. Potential Prototype Pollution via Query Parameters

Code:

typescript

```
19 const icons = apiUrl.searchParams.get('icons')?.split(',')
```

- **Line 19:**
 - The query parameter `icons` is extracted directly from the user input and split into an array. If not sanitized, it could allow **Prototype Pollution** through specially crafted inputs.

Mitigation:

- Validate and sanitize query parameters before processing them.
 - Consider using stricter parsing logic or schema validation.
-

Summary Table of Vulnerable Lines

Vulnerability	Lines	Issue
SSRF via URL Parsing	9-11, 18	Unvalidated user input in URL parsing
Prototype Pollution via Collections	12-15	Untrusted dynamic object access
Information Leakage in Dev Mode	23-31	Sensitive data in logs
External Fetch without Error Handling	32-36	No error handling for API calls
DoS via Unrestricted API Calls	34	No rate limiting on external fetch

Vulnerability	Lines	Issue
Prototype Pollution via Query Parameters	19	Unvalidated query parameter input

Recommended Fixes

1. **Input Validation & Sanitization:**
 - Validate `url`, `collectionName`, and query parameters.
2. **Error Handling:**
 - Add `try-catch` blocks around external API calls.
3. **Rate Limiting / Caching:**
 - Use caching or rate limiting for API calls.
4. **Secure Logging:**
 - Ensure that logs in development mode do not leak sensitive information.

Implementing these improvements will significantly enhance the security of your code.

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Here is an in-depth security analysis of your TypeScript code, with mappings to the **OWASP Top 10 Web Application Security Risks**. Each identified issue is mapped to one or more OWASP categories, helping you understand its significance and required mitigations.

1. Insufficient Input Validation for URL Parsing

Code:

```
typescript

9  const url = ctx.node.req.url
10 if (!url) return
```

```
...
18 const apiUrl = new URL(basename(url), apiEndPoint)
```

Issue:

- The `url` input is taken directly from the request without validation or sanitization and used to create a new `URL` object.
- **Risk:** This can result in **Server-Side Request Forgery (SSRF)** if an attacker sends crafted requests to internal services.

OWASP Mapping:

- **A10:2021 – Server-Side Request Forgery (SSRF):** The application fetches remote resources without validating the URLs, potentially leading to SSRF attacks.
- **A05:2021 – Security Misconfiguration:** If the external URL is misused or manipulated, it could expose sensitive services.

Mitigation:

- Validate and restrict the allowed URLs (use allowlists).
- Ensure that internal URLs (e.g., `localhost`, private IPs) are blocked.

2. Prototype Pollution via Dynamic Access to Collections

Code:

```
typescript

12 const collectionName = ctx.context.params?.collection?.replace(/\.json$/, '')
13 const collection = collectionName
14 ? await collections[collectionName]?.()
15 : null
```

Issue:

- The `collectionName` is user-controlled and directly used to access properties within the `collections` object.

- **Risk:** This could lead to **Prototype Pollution**, where attackers manipulate the application's internal structure by injecting unexpected object properties.

OWASP Mapping:

- **A01:2021 – Broken Access Control:** Allowing unchecked dynamic access to internal objects can result in unauthorized access.
- **A08:2021 – Software and Data Integrity Failures:** Manipulating internal data structures through uncontrolled parameters is a data integrity issue.

Mitigation:

- Use strict input validation on `collectionName`.
 - Use an **allowlist** of valid collection names to avoid unauthorized object access.
-

3. Information Leakage via Logging in Development Mode

Code:

```
typescript

23 if (import.meta.dev) {
24   if (collectionName && !warnOnceSet.has(collectionName) && apiEndPoint ===
DEFAULT_ENDPOINT) {
25     consola.warn([
26       `[Icon] Collection \`${collectionName}\` is not found locally`,
27       `We suggest to install it via \`${npm i -D @iconify-json/${collectionName}\`
to provide the best end-user experience.`
28     ].join('\n'))
29     warnOnceSet.add(collectionName)
30   }
31 }
```

Issue:

- The code logs internal state (e.g., collection names) in development mode. This could lead to **information leakage** if accidentally exposed in production or if development logs

are accessible.

OWASP Mapping:

- **A01:2021 – Broken Access Control:** If internal state or debug logs are available inappropriately, it could provide unauthorized access to sensitive information.
- **A09:2021 – Security Logging and Monitoring Failures:** Logs might contain too much information, aiding attackers.

Mitigation:

- Ensure that sensitive logs are disabled in production.
 - Use environment variables to control the logging level effectively.
-

4. External API Fetch without Proper Error Handling

Code:

typescript

```
32 if (options.fallbackToApi) {
33   consola.debug(`[Icon] fetching ${icons || []}.map(i => '' + collectionName +
34     ':' + i + '').join(',') from iconify api`)
35   const data = await $fetch(apiUrl.href)
36   return data
37 }
```

Issue:

- The `$fetch` method makes an external request without proper error handling or input validation.
- **Risk:** If the external API returns an unexpected or malicious response, it may introduce **Remote Code Execution (RCE)** or cause the application to fail.

OWASP Mapping:

- **A03:2021 – Injection:** External API responses could introduce dangerous inputs if not properly validated.

- **A08:2021 – Software and Data Integrity Failures:** Unchecked responses from external systems pose a risk to data integrity and application security.

Mitigation:

- Add `try-catch` blocks to handle network errors.
 - Use **schema validation** to verify that the response matches the expected structure.
-

5. Lack of Rate Limiting for API Calls (DoS Risk)

Code:

```
typescript

34 const data = await $fetch(apiUrl.href)
```

Issue:

- The `$fetch` call is triggered without any rate limiting or caching. This could allow attackers to **flood the external API** with requests, causing **Denial of Service (DoS)**.

OWASP Mapping:

- **A06:2021 – Vulnerable and Outdated Components:** Over-reliance on external APIs without proper rate limiting exposes the system to DoS attacks.
- **A04:2021 – Insecure Design:** Not implementing rate limiting or caching is a design flaw.

Mitigation:

- Introduce **rate limiting** mechanisms to prevent excessive calls.
 - Use **caching** to store previous API responses when appropriate.
-

6. Potential Prototype Pollution via Query Parameters

Code:

```
typescript
```

```
19 const icons = apiUrl.searchParams.get('icons')?.split(',')
```

Issue:

- Query parameters are directly accessed from the URL without sanitization. Malicious input could exploit **Prototype Pollution**, leading to unexpected behavior or system compromise.

OWASP Mapping:

- **A03:2021 – Injection:** Query parameters might inject malicious content.
- **A05:2021 – Security Misconfiguration:** Using unvalidated input can lead to various misconfigurations or exploitation.

Mitigation:

- Use **input sanitization** and validate the query parameters.
- Apply strict parsing to ensure the `icons` parameter only contains expected values.

Summary of Vulnerabilities and OWASP Mapping

Vulnerability	Lines	OWASP Risk Category
SSRF via URL Parsing	9-11, 18	A10:2021 – Server-Side Request Forgery (SSRF), A05:2021 – Security Misconfiguration
Prototype Pollution via Collections	12-15	A01:2021 – Broken Access Control, A08:2021 – Software and Data Integrity Failures
Information Leakage in Development Mode	23-31	A01:2021 – Broken Access Control, A09:2021 – Security Logging and Monitoring Failures
External API Fetch without Error Handling	32-36	A03:2021 – Injection, A08:2021 – Software and Data Integrity Failures
Lack of Rate Limiting (DoS Risk)	34	A06:2021 – Vulnerable and Outdated Components, A04:2021 – Insecure Design

Vulnerability	Lines	OWASP Risk Category
Prototype Pollution via Query Parameters	19	A03:2021 – Injection, A05:2021 – Security Misconfiguration

Conclusion and Recommendations

The code contains several potential vulnerabilities that map to multiple OWASP Top 10 categories. Below are the **key recommendations**:

1. **Validate all user inputs**, including URLs, collection names, and query parameters.
2. **Implement error handling** using `try-catch` around network calls.
3. **Restrict internal object access** by validating dynamic keys like `collectionName`.
4. **Use rate limiting and caching** to prevent DoS attacks.
5. **Secure logging**: Ensure no sensitive information leaks in production logs.

By addressing these issues, you can make the application more resilient to modern security threats.

For each identified vulnerability, provide an exploit example that an attacker could use. Include a step-by-step explanation of how the exploit works, the necessary conditions or setup, any potential limitations or obstacles an attacker might face, and the likelihood of this attack happening. Ensure the exploits are specific to the vulnerability type provided.

Here are potential exploits for each identified vulnerability in the code, along with step-by-step explanations, conditions required for exploitation, potential limitations, and likelihood of occurrence:

1. SSRF via URL Parsing

Code:

typescript

```
18 const apiUrl = new URL(basename(url), apiEndPoint)
```

Exploit Example:

- **Attack:** An attacker sends a request with a crafted URL like `/internal/admin`.
- **Steps:**
 1. Attacker sends `GET /internal/admin` to the server.
 2. The server constructs the URL: `new URL(basename('/internal/admin'), DEFAULT_ENDPOINT)`.
 3. If `DEFAULT_ENDPOINT` is an internal API (e.g., `http://127.0.0.1`), the request accesses sensitive internal services.
- **Conditions:**
 - The attacker needs control over the `ctx.node.req.url`.
 - The server must process URLs with `apiEndPoint` pointing to sensitive resources.
- **Limitations:**
 - If URL validation exists, SSRF attempts will fail.
- **Likelihood:**
 - **Moderate:** If `ctx.node.req.url` is not validated and external endpoints are used, this attack is likely.

2. Prototype Pollution via Collections

Code:

typescript

```
14 const collection = collectionName ? await collections[collectionName]?.() : null
```

Exploit Example:

- **Attack:** An attacker injects malicious properties into the `collections` object.

- **Steps:**
 1. Attacker sends a request with `collectionName` set to `__proto__`.
 2. The server processes `collections["__proto__"]`, potentially adding properties to `Object.prototype`.
 3. If `Object.prototype` is polluted (e.g., adding `toString` that executes malicious code), it affects all object operations in the app.
 - **Conditions:**
 - Attacker must control `ctx.context.params?.collection`.
 - `collections` should not have input validation or restrictions.
 - **Limitations:**
 - Prototype pollution depends on how objects are used in the codebase.
 - **Likelihood:**
 - **High:** This attack is common when dynamic object access is unchecked.
-

3. Information Leakage in Development Mode

Code:

```
typescript
```

```
26 `[Icon] Collection \`${collectionName}\` is not found locally`
```

Exploit Example:

- **Attack:** An attacker reads sensitive logs to gather information about available collections.
- **Steps:**
 1. Server runs in development mode (`import.meta.dev` is true).
 2. Attacker requests a non-existent collection (e.g., `/icons/unknown.json`).
 3. The log leaks details about `collectionName` and other internal settings.
- **Conditions:**

- The application must run in development mode and expose logs.
 - **Limitations:**
 - Attackers may have limited access to logs unless exposed through other misconfigurations.
 - **Likelihood:**
 - **Low:** This is less likely in production, but in development, it can aid attackers in reconnaissance.
-

4. External API Fetch without Proper Error Handling

Code:

```
typescript

34 const data = await $fetch(apiUrl.href)
```

Exploit Example:

- **Attack:** An attacker manipulates the API endpoint to deliver a malicious payload.
 - **Steps:**
 1. Attacker provides an endpoint in the query string like `?icons=payload`.
 2. `$fetch` fetches the payload (e.g., a script injecting malicious code).
 3. The malicious response is processed or rendered on the client without validation.
 - **Conditions:**
 - The server must fetch from attacker-controlled URLs without schema validation.
 - **Limitations:**
 - If strict CORS policies or response validation exists, this attack is mitigated.
 - **Likelihood:**
 - **Moderate:** Exploitable when `$fetch` fetches untrusted endpoints.
-

5. Lack of Rate Limiting for API Calls (DoS Risk)

Code:

typescript

```
34 const data = await $fetch(apiUrl.href)
```

Exploit Example:

- **Attack:** An attacker floods the API with a high number of requests, exhausting server or third-party resources.
- **Steps:**
 1. Attacker identifies the route handling icon requests.
 2. Using a botnet or script, sends thousands of requests (e.g., `/icons/some-icon.json`).
 3. The server or the third-party API becomes overwhelmed, causing a Denial of Service (DoS).
- **Conditions:**
 - The attacker must discover the endpoint and its behavior.
 - There must be no rate limiting or caching in place.
- **Limitations:**
 - If rate limiting or caching exists, the attack's impact is reduced.
- **Likelihood:**
 - **High:** Without rate limiting, this is a likely attack vector.

6. Prototype Pollution via Query Parameters

Code:

typescript

```
19 const icons = apiUrl.searchParams.get('icons')?.split(',')
```

Exploit Example:

- **Attack:** An attacker injects malicious properties via query parameters.
- **Steps:**
 1. Attacker sends a request with query parameters like `?icons=__proto__:maliciousValue`.
 2. The server parses the query, and `split(',')` processes `__proto__`.
 3. This modifies the prototype of objects in the application.
- **Conditions:**
 - Query parameters must directly map to objects without sanitization.
- **Limitations:**
 - Application behavior after prototype pollution determines the exploit's impact.
- **Likelihood:**
 - **Moderate:** This attack is possible if query parameters are not sanitized.

Summary Table

Vulnerability	Exploit Example	Conditions	Limitations	Likelihood
SSRF via URL Parsing	Exploit internal services by manipulating URLs (e.g., <code>/internal/admin</code>).	Requires unvalidated URL input.	Blocked by input validation or allowlists.	Moderate
Prototype Pollution via Collections	Inject <code>__proto__</code> or similar properties to manipulate application logic.	Requires dynamic object access.	Limited if object usage is safe.	High
Information Leakage in Dev Mode	Read sensitive logs to discover internal collections or endpoints.	Requires access to development logs.	Unlikely in production.	Low

Vulnerability	Exploit Example	Conditions	Limitations	Likelihood
External API Fetch without Validation	Inject malicious payloads via external API responses.	Requires control over API endpoint.	Schema validation reduces risks.	Moderate
Lack of Rate Limiting (DoS Risk)	Flood the API with requests to cause downtime.	Endpoint must allow uncontrolled requests.	Rate limiting or caching mitigates impact.	High
Prototype Pollution via Query Params	Inject <code>__proto__</code> or similar properties via query parameters like <code>?icons=__proto__:maliciousValue</code> .	Requires unsanitized query processing.	Reduced by strict input parsing.	Moderate

By addressing these potential exploits with robust input validation, logging restrictions, rate limiting, and prototype protection mechanisms, you can mitigate the associated risks effectively.